For this example, the ground truth final output y is $y =$ "`1403,5725,2304,2075,1872`" (i.e., the model reached the correct final output in the example above).

For the sample given above, we have that

$$
\begin{aligned}
x_0^M &= x_0 = \text{“}\texttt{83,48,14,98,25}\text{''} \\
x_1^M &= \text{“}\texttt{83,48,14,98,25 || 48,83,98,14,25 }\text{''} \\
&\vdots \\
x_5^M &= \text{“}\texttt{83,48,14,98,25 || 48,83,98,14,25 || 48,131,229,25,14 ||} \\
&\qquad \texttt{229,131,48,25,39 || 1872,3275,5725,2304,229 ||} \\
&\qquad \texttt{1403,5725,2304,2075,1872}\text{''}
\end{aligned}
$$

Next, to construct the data for fine-tuning the supervisor model, we used the special token "`#|>`" to separate the model's CoT steps x_i^M from the ground truth output y. This results in the following 6 training datapoints for the supervisor model:

1. "`83,48,14,98,25 #|> 1403,5725,2304,2075,1872`"
2. "`83,48,14,98,25|| 48,83,98,14,25 #|> 1403,5725,2304,2075,1872`"

   ⋮

5. "`83,48,14,98,25|| 48,83,98,14,25 || 48,131,229,25,14 || 229,131,48,25,39 || 1872,3275,5725,2304,229 || 1403,5725,2304,2075,1872 #|> 1403,5725,2304,2075,1872`"

The above procedure allows us to obtain fine-tuning data for supervisor models separately for each of the 5 different LLMs, $\{\text{LLM}_1, \text{LLM}_2, \ldots, \text{LLM}_5\}$. Next, we train a separate GPT-2 model for each of the 5 different base LLMs.

C.1.4 Estimating the information-gain

Having trained the supervisor model on the data generated above, we evaluate the information-gain on a held-out dataset split. Given a datapoint (x_i^M, y) in the evaluation split, we can estimate the sample-wise information-gain at step i as follows:

- Suppose that the model generation at step $i-1$, x_{i-1}^M is tokenised as $(t_1, \ldots, t_{n_{i-1}})$ and similarly that x_i^M is tokenised as $(t_1, \ldots, t_{n_i})$. Likewise, suppose that the true output y is tokenised as $(t_1^*, \ldots, t_k^*)$ and we use $< s >$ to denote the separator token (i.e. `#|>` above).
- Then, to estimate the sample-wise for this datapoint, we estimate the difference:

$$
\begin{aligned}
&\frac{1}{k}\sum_{j=1}^{k} \log p(t_j^* \mid (t_1, \ldots, t_{n_i}, < s >, t_1^*, \ldots, t_{j-1}^*)) \\
&\qquad - \frac{1}{k}\sum_{j=1}^{k} \log p(t_j^* \mid (t_1, \ldots, t_{n_{i-1}}, < s >, t_1^*, \ldots, t_{j-1}^*)).
\end{aligned}
$$

  Here, the supervisor model is trained to estimate the above conditional and therefore we use it to estimate the difference above.

Finally, to estimate the aggregate information-gain (instead of the sample-wise information-gain), we simply compute the average sample-wise gain over the evaluation data split.

C.1.5 Additional results

In Figures 5 - 7, we present the sample-wise trajectories for 15 randomly chosen prompts leading to incorrect final answers, for the different baselines and LLMs under consideration. Here, any significant drop in the

plotted value at a given step could be seen as an indication of an incorrectly executed sub-task. Recall that in our setup, in LLM_i, the CoT step i is executed incorrectly with some probability whereas all other steps are always executed correctly.

Firstly, Figure 5 presents sample-wise information-gain for our method for the five different LLMs. Here, we see that the sample-wise information remains high up until the incorrect step, at which point the information-gain sharply decreases. This suggests that sample-wise information-gain is sensitive to the specific point where the Chain of Thought goes wrong, making it effective at locating reasoning errors.

For the ORM and Math-Shepherd baselines in Figures 6 and 7, we observe that for all LLMs except LLM_3, the plotted metrics drop at the incorrect step. However, for LLM_3, we observe that ORM's probability of correctness drops at step 2 even though the error occurs at step 3. This occurs because, in our setup, the correctness of step 3 is determined directly from the output of step 2. Specifically, recall that in LLM_3, step 3 is executed incorrectly if and only if the output of step 2, z_2, has its second component greater than 150, i.e. $z_2[2] > 150$. Therefore, ORM becomes confident after the second step if a CoT is going to lead towards the correct final answer or not.

Similarly, for Math-Shepherd in Figure 7, we observe that the proportion of correct completions remains 0 for LLM_3. This is because for all trajectories plotted, the output of step 2, z_2, has its second component greater than 150 and therefore the final answer is incorrect regardless of which step we begin the completions from.

C.2 Arithmetic Operations on Llama-3-8b

For this experiment, the prompts used to collect the data follow a specific structure. Each prompt contains two real examples followed by a query with newly sampled values for x and y. The format of the prompt is as follows:

```
x = 23, y = 51. Please calculate the following:
1. 3x
2. 2y
3. 3x + 2y
Answer:
1. 3x = 69
2. 2y = 102
3. 3x + 2y = 171

x = 35, y = 60. Please calculate the following:
1. 3x
2. 2y
3. 3x + 2y
Answer:
1. 3x = 105
2. 2y = 120
3. 3x + 2y = 225

x = {x}, y = {y}. Please calculate the following:
1. 3x
2. 2y
3. 3x + 2y
Answer:
```

In the third section, the values of x and y are randomly sampled from a uniform distribution over the range $[1, 10^5)$.

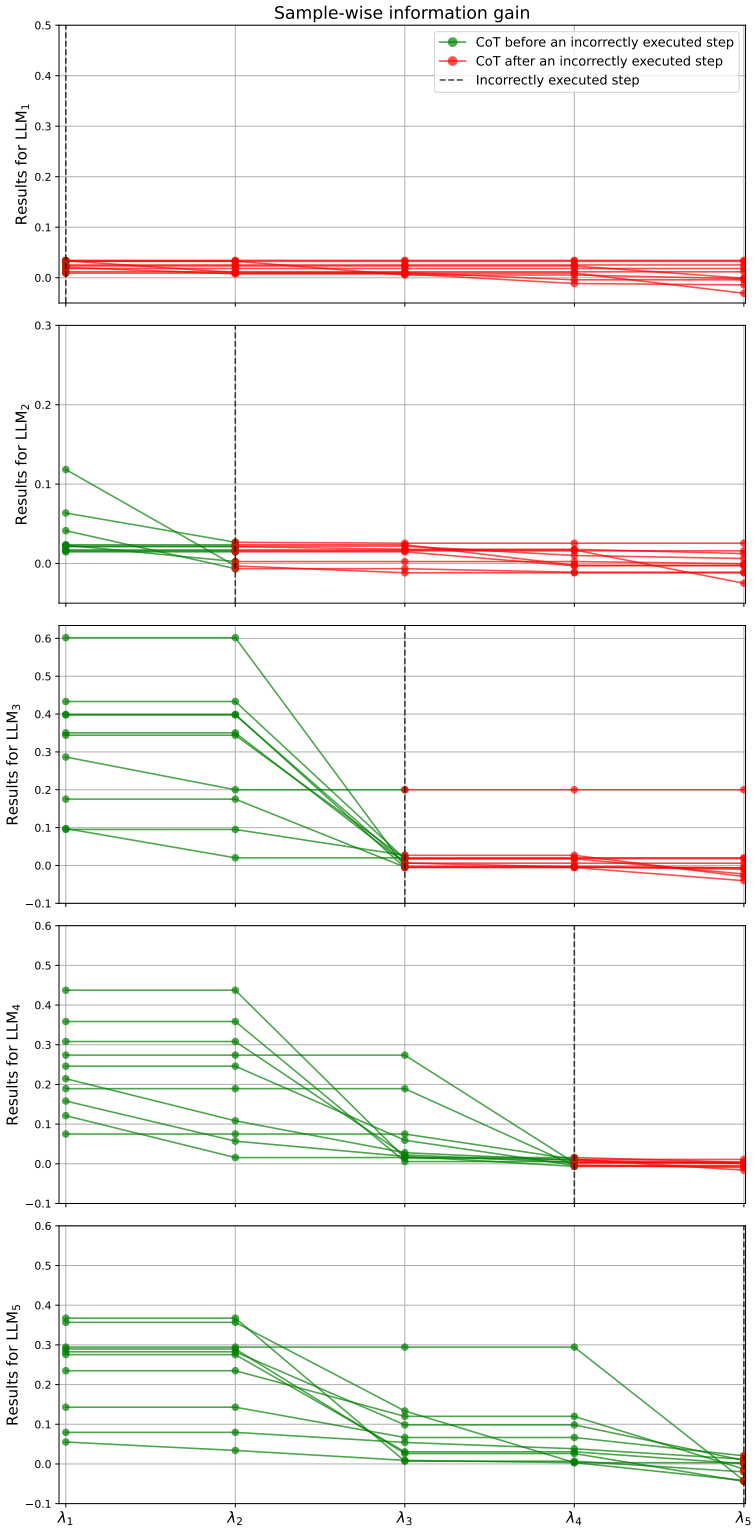

Figure 5 Toy data results: Sample-wise information-gain trajectories for 15 randomly chosen prompts with wrong final answers.